

Contents

1	How To read a scientific article (09.04)	3
1.0.1	Begriffe	3
2	Pseudocode schreiben (16.04)	5
2.1	Pseudo code	5
2.2	K-mer counting exercise	6
3	Vortrag	9
3.1	Minimizer-space de Bruijn graphs: Whole-genome assembly of long reads in minutes on a personal computer.	9
3.1.1	Eckdaten	9
3.1.2	Erarbeitung	9
3.1.3	Was ist neu?	9
	Sources	11
	Index	11
	Glossary	12
	List of Figures	12
	List of Tables	13

Chapter 1

How To read a scientific article (09.04)

1.0.1 Begriffe

- : sind relevant für:

- assembly
- error correction
- alignment mapping
- genome comparison
- ...
-
-

Benutzt für:

- counting
 - indexing
 - de bruijn Graphen
- **Canonical k-mers:** Wenn die Leserichtung unbekannt ist schaut man sich beide Richtungen an (das k-mer und das reverse Komplement) und nimmt eines davon als grundsätzlichen Repräsentanten des Paares (das ist dann das canonical). Canonical ist nach einem Kriterium ausgewählt, bspw. das was lexikographisch kleiner ist.
 - **Minimizer:** Wenn es zu viele k-mere gibt, macht man ein subsampling.
Ds geht z.B durch : Man nimmt von den k-meren in einem w-langen Bereich der Sequenz das lexikographisch kleinste k-mer. Oder auch durch Bottom Sketch (kleinste x% der k-mere).

Tipp

Man sollte das Velvet Paper für de Bruijn auf jeden Fall lesen, weil die eine gute Erklärung für de Bruijn Graphen haben, die in späteren Papers nicht nochmal so ausführlich gegeben wird.

Mein Thema: Minimizer-space de Bruijn graphs: Whole-genome assembly of long reads in minutes on a personal computer.

Chapter 2

Pseudocode schreiben (16.04)

Tipps

- In Latex!
- möglichst an Mathematische Notationen halten
- schön sschreiben, deswegen benutzen wir ja Latex! Also nicht x_i sondern x_i
- mit = aufpassen! Eher ":@" benutzen für Definitionen und für Folgerungen Pfeile

2.1 Pseudo code

To present an algorithm, you can either first describe it in prose and then refer to a summary in pseudo code, or you first refer to the pseudo code, followed by an explanation of the comprised steps. Algorithm 1 on page 6 gives an example of pseudo code.

Note that all terms, data types and meanings of variables need to be clear – either from the main text or specified in the pseudo code. For instance, write “rooted tree T ” instead of just “ T ”. Properly specify the input and output, even if you think it is clear from the main text, because often it is not, and even if so, pseudo code (like tables and figures) should be self-contained.

You can point to individual lines or blocks composed of several lines. This is also useful when you discuss the run time complexity of an algorithm, e.g., the for-loop in lines 11–12.

Level of detail. Be only as detailed and technical as necessary to not obfuscate the procedure but still clarify all relevant aspects. Whether a step needs to be broken into sub-steps often depends on the context. If its implementation is not obvious or a naive implementation would ruin the overall run time, break it down. In particular, breaking down trivial things like the selection of a minimum from a list or summing up its values, would only hinder the reader to understand the algorithm. In such cases, use standard mathematical formalism, such as $\min\{\dots\}$ or $\Sigma \dots$.

Notation. In general, when presenting algorithms, mathematical notation is preferable to programming notation, because it is standardized and commonly known by computer scientists. Feel free to combine it with prose, like in line 11 of Algorithm 1. In particular, there is no need to use ascii-art such as “ x_i ” rather than “ x_i ”, or “ $x*y$ ” rather than “ $x \cdot y$ ”. Use “ $\leftarrow$ ” for assignments to distinguish it from equality (“ $=$ ”) or mathematical definitions (“ $:=$ ”). The latter might only be needed to *specify* a value but not to *assign* it to any variable, see, e.g. line 12.

Tipp

Zeilennummern braucht man immer! Sonst macht es wenig Sinn im Text auf eine bestimmte Zeile zu verweisen.

Algorithm 1: Fitch-Hartigan for Zero-One Instances**Input:** A rooted tree $T = (V, E)$ with each leaf $l \in V$ labeled with $L(l) \subseteq \mathcal{C}$.**Output:** A labeling L with minimal Wagner parsimony weight $W(T, L)$.

```

1 foreach character  $c \in \mathcal{C}$  do
    /* Bottom-up phase */
2   foreach leaf  $l$  do
3      $VU(l) \leftarrow \{L_c(l)\}$ 
4      $VL(l) \leftarrow \emptyset$ 
5   foreach unlabeled node  $u$  whose children  $v_1, \dots, v_{l(u)}$  are labeled do
6     for  $b \in \{0, 1\}$  do  $k(b) \leftarrow |\{v_i \mid b \in VU(v_i)\}|$ 
7      $K \leftarrow \max\{k(0), k(1)\}$ 
8      $VU(u) \leftarrow \{b \mid k(b) = K\}$ 
9      $VL(u) \leftarrow \{b \mid k(b) = K - 1\}$ 
    /* Top-down refinement */
10  assign any  $b \in VU(r)$  to the root node  $r$ :  $L_c(r) := b$ 
11  foreach unrefined node  $v$  whose parent node  $u$  is already refined to  $L_c(u) = a$  do
12    refine the labeling of  $v$  to  $L_c(v) \leftarrow b$ , with any  $b \in B(v, a)$ :
        
$$B(v, a) := \begin{cases} \{a\} & \text{if } a \in VU(v), \\ \{a\} \cup VU(v) & \text{if } a \in VL(v), \\ VU(v) & \text{otherwise.} \end{cases}$$

13 return  $L$ 

```

2.2 K-mer counting exercise

Vorgehen

Wenn man in der Sequenz einem N begegnet, schmeißt man alle k-mere die das N enthalten weg und macht hinter dem N weiter.

First, read in sequence and return all canonic k-mers of a sequence. Oee rauch: Leere Liste initialisieren, dann

Algorithm 2: Reading in k-mers**Input:** A string of length n , an integer value for k **Output:** A list L of all canonic k -mers contained in the input sequence

```

1  $L \leftarrow \emptyset$ ;
2  $index \leftarrow 0$ ;
3 while  $index < n - k$  (end of sequence not reached) do
4    $kmer_c \leftarrow$  letters from position  $index$  to  $index + k$ ;
5   if  $kmer_c$  enthält Zeilenumbruch then
6     Zeilen verbinden damit es weitergehen kann
7   if char "N" in  $kmer_c$  then
8      $index \leftarrow index + (pos("N") \text{ in } kmer_c)$ ;
9   else
10    Find canonic  $k$ -mer;
11     $index ++$ ;
12 return  $L$ 

```

alle Zeilen durchgehen, enn es mit > (Header Zeile) beginnt neues x leer initialisieren und das reverse Komplement xwelle sirekt mit leer initialisieren und zur nächsten Zeile springen. Sonst (wenn es eine Sequenzzeile ist) fpr jeden character: Wenn N, dann mache x leer und mache xwelle auch leer und skippe zum nächsten character . Sonst append character zu x un dfüge komplement von c zu xwelle hinzu. Wenn l' die länge von X schon k ist, dann ist $\min\{x, xwelle\}$ das kanonische kmer und man fügt das canonischt zur Liste hinzu. Dann löscht man in x und xwelle den erste charcater und macht mit den Schleifen weiter, damit man nicht alles neu einlesen muss.

Algorithm 3: *\caption*

Input: $\backslash KwIn\{input\}$
Output: $\backslash KwOut\{output\}$
Data: $\backslash KwData\{datainput\}$
Result: $\backslash KwResult\{dataoutput\}$

Algorithm 4: Reading in k-mers - k-mer counting exercise

Input: A multiple fasta file *file*, an integer value for *k*, a threshold *c*
Output: number of all canonic *k*-mers contained in the input sequences

```

1  $L \leftarrow \emptyset$ ; /* initialize empty list */
2  $kmer_{comp} \leftarrow ""$ ; /* initialize complementary k-mer */
3  $kmer \leftarrow ""$ ; /* initialize k-mer */
4  $hashmap(key, value) \leftarrow \emptyset$ ; /* for the Threshold */
5 foreach line in file do
6   if line contains ">" then /* if a new sequence begins */
7      $kmer_{comp} \leftarrow ""$ ;
8      $kmer \leftarrow ""$ ;
9   else
10    foreach Character char in line do
11       $char \leftarrow uppercase(char)$ ;
12      if char = "N" then /* "N" in kmer */
13         $kmer_{comp} \leftarrow ""$ ;
14         $kmer \leftarrow ""$ ;
15      else
16         $kmer$  append char;
17         $kmer_{comp}$  prepend complement of char;
18        if length( $kmer$ ) = k then
19           $canon \leftarrow \min\{kmer_{comp}, kmer\}$ ;
20          if exists  $hashmap[canon]$  then /* if kmer already in hasmap */
21             $old \leftarrow hashmap[canon]$ ;
22             $hashmap[canon] = old + 1$ ; /* update count */
23          else /* otherwise add with count 1 */
24             $hasmap[canon] = 1$ 
25          if  $hashmap[canon] = c$  then /* if Threshold reached */
26             $L$  add canon
27           $kmer$  remove first element;
28           $kmer_{comp}$  remove last element;
29 return length( $L$ )

```


Chapter 3

Vortrag

3.1 Minimizer-space de Bruijn graphs: Whole-genome assembly of long reads in minutes on a personal computer.

3.1.1 Eckdaten

- **Länge:** 20-45 min

Ekim, Barış, Bonnie Berger, and Rayan Chikhi. “Minimizer-space de Bruijn graphs: Whole-genome assembly of long reads in minutes on a personal computer.” *Cell systems* 12.10 (2021): 958-968.

3.1.2 Erarbeitung

Was macht der überhaupt?

- Whole-genome assembly

Wieso wurde der entwickelt?

- DNA-Sequenzierung entwickelt sich weiter und die daraus entstehenden Reads werden länger und die zusammenzusetzen ist schwierig
 - DNA sequencing data continue to progress toward longer reads with increasingly lower sequencing error rates
 - High-throughput DNA sequencing
- improvements in speed and memory usage for human genome assembly and metagenome assembly
 - achieves orders-of-magnitude improvement in both speed and memory usage over existing methods without compromising accuracy
 - It improves the efficiency of [pangenome graph construction](#)
 - here has traditionally been a trade-off between algorithmic efficiency and loss of information, at least during the initial sequence-processing steps

3.1.3 Was ist neu?

- [minimizer-space sequencing analysis](#)
- expanding the alphabet of DNA sequences to [atomic tokens](#) made of fixed-length words
- instead of building an assembly over sequence bases—the standard approach that for clarity we refer to as base space—newly performs assembly in [atomic space](#) and later converts it back to base-space assemblies
- Specifically, each read is initially converted to an ordered sequence of its

Welche Datenstrukturen und Strategien benutzt er?

Was macht er besser als andere?

Was macht er schlechter als andere?

Aufgekommene Fragen

- Wie ist es möglich aus Minimizen die ursprüngliche Sequenz wieder zu rekonstruieren?
- Werden Window-Minimizer oder globale Minimizer verwendet?
- wie werden die Minimizer ausgesucht?
- wie wird festgestellt, ob eine Sequenz high oder low error ist?

Begriffe

atomic tokens -. [9](#)

minimizer-space sequencing analysis -. [9](#)

pangenome graph construction -. [9](#)

List of Figures

List of Tables